

Teoría de Algoritmos

(75.29) Curso Buchwald - Genender

Trabajo Práctico 2:

Programación Dinámica For The Win

---

12 de octubre de 2026

Lucio Romero Mascaró
113091

Valentín Raunic
114081

Índice

1. Resumen **3**

2. El Problema **4**

2.1. Introducción 4

2.2. Planteamiento 4

2.3. Consecuencias de la optimalidad 5

3. Teoría Programación Dinámica **6**

3.1. Programación Dinámica aplicada al problema 7

3.2. Pensamiento y utilidades 9

4. Diseño del Algoritmo **10**

4.1. Lógica 10

4.2. Código 11

5. Analizando el Algoritmo **13**

5.1. Complejidad 16

5.2. Demostración de la Optimalidad 16

5.2.1. Impacto de la Variabilidad de los datos 17

5.3. Experimentos 18

6. Conclusión **22**

Bibliografía **22**

1. Resumen

Se presenta un resumen del Trabajo Práctico 2, detallando los temas a tratar para ofrecer una lectura fluida, clara y precisa que permita comprender el desarrollo en su totalidad.

En primer lugar, se va a hablar sobre el problema propuesto, cómo fue el planteo de la resolución y cómo se logró llegar a que ésta misma pueda ser óptima para cualquier caso posible.

Se detalla la teoría necesaria sobre los *algoritmos de Programación Dinámica*, enfocándose en lo más básico de la teoría, entre ella la *Ecuación de Recurrencia*, la forma de ver los subproblemas, la toma de decisiones y la *técnica de memoization*. Para ver estos aspectos aplicados, se incluye un breve ejemplo de la idea en general y su analogía a conceptos en el mundo real (alcance de metas y aprender de los errores).

Por otro lado, se detalla el diseño del algoritmo implementado en *pseudocódigo*, explicando anteriormente la lógica, cómo fue buscada y cómo fue pensada esa resolución. Adjuntándose al final la implementación realizada.

Se realiza un *análisis* del algoritmo planteado, mediante el estudio de la complejidad y una demostración matemática para validar la optimalidad del algoritmo, de forma clara y con ejemplos propuestos. Se incorporan múltiples experimentos empíricos de forma tal que se pueda visualizar gráficamente las medidas de tiempo sobre cierta cantidad de casos.

Finalmente, se expone una conclusión sobre lo aprendido, la dificultad del problema, y la particularidad de estos algoritmos y su pensamiento.

2. El Problema

Mateo tiene 7 años. Los mismos años que tenía Sophia cuando comenzaron a jugar al juego de las monedas. Eso quiere decir que Mateo también ya aprendió sobre algoritmos greedy, y lo comenzó a aplicar. Esto hace que ahora quién gane dependa más de quién comience y un tanto de suerte.

Esto no le gusta nada a Sophia. Ella quiere estar segura de ganar siempre. Lo bueno es que ella comenzó a aprender sobre programación dinámica. Ahora va a aplicar esta nueva técnica para asegurarse ganar siempre que pueda.

El juego de las monedas consiste en disponer una fila de n monedas, de diferentes valores. En cada turno, un jugador debe elegir alguna moneda. Pero no puede elegir cualquiera: *sólo puede elegir o bien la primera de la fila, o bien la última.* Al elegirla, la remueve de la fila, y le toca luego al otro jugador, quien debe elegir otra moneda siguiendo la misma regla. Siguen agarrando monedas hasta que no quede ninguna. Quien gane será quien tenga el mayor valor acumulado (por sumatoria).

Para transformar esta situación en un problema algorítmico concreto, el desafío es el siguiente: Hacer un análisis del problema, y proponer un algoritmo de Programación Dinámica que obtenga la solución óptima al problema planteado: Dada la secuencia de monedas m_1, m_2, ..., m_n, sabiendo que Mateo siempre elegirá la moneda más grande para sí en su respectivo turno, indicar qué monedas debe ir eligiendo Sophia para si misma, de tal forma que Sophia gane siempre que pueda, y que siempre alcance el *mayor valor acumulado posible*. Considerar que Sophia siempre comienza.

2.1. Introducción

Tranquilamente, se puede observar que los objetivos son dos: que *Sophia gane siempre que pueda* y que *alcance el mayor valor acumulado posible*. A eso sumémosle que vamos a saber que Mateo siempre va a elegir la moneda de mayor valor en su turno.

Por último, lo más importante es que la elección va a ser si o sí entre 2 monedas y el valor entre ellas. Se puede preguntar:

- ¿Cómo se podría hacer que eligiendo x moneda (una entre esas dos), no haya otra moneda próxima (mayor a x) que Mateo la elija en el turno siguiente?
- ¿Cómo es posible crear un algoritmo de Programación Dinámica en el cual sea óptimo para cualquier caso de monedas? ¿Qué es un algoritmo de Programación Dinámica y qué significa que sea óptimo?
- ¿Qué casos van a ser que Sophia pierda? ¿Es posible que Sophia gane siempre?

Una vez propuesto eso, sigamos pensando para ahora empezar con el planteo.

2.2. Planteamiento

Hacer que gane siempre que pueda Sophia, y tenemos la suerte también de que Sophia tiene *visión* por sobre todas las monedas y *no tiene el sesgo* de elegir siempre la mayor. Entonces, cómo

podemos aconsejar a Sophia de tal forma que si siempre hace lo que le hayamos dicho, gane siempre que pueda y se quede con el mejor valor acumulado posible?

Primero que nada, hay que aclarar que si Sophia consigue en todos los casos el mejor valor acumulado posible $\Longrightarrow$ va a ganar siempre que pueda. La solución que se puede ocurrir primero es que entre las dos monedas a elegir agarre siempre la mayor en valor. Eso va a garantizar que siempre termine con una suma mayor que Mateo. O no?

Pero las monedas que elige Mateo, ¿van a llegar a repercutir mucho en el problema?

Para que Sophia gane, tiene que tener la mayor cantidad de valor acumulado comparado con Mateo, entonces si Sophia agarra por cada par siempre la mayor, Sophia va a competir con las monedas de mayor valor que agarre Mateo. La manera *avariciosa* de Sophia, copiándole a Mateo, la va a llevar por mal camino.

Simple contraejemplo: la fila de monedas son $[5, 100, 1, 2]$. Turno Sophia, entre $[5, 2]$ agarra la de mayor valor $[5]$. Turno Mateo, entre $[100, 2]$ agarra la de mayor valor tambien $[100]$. Ahora sin importar la que agarre Sophia, va a terminar *perdiendo y sin el mayor valor acumulado.*

La manera avariciosa, manera *Greedy*, pierde la visión del "Big Picture" del problema. Ahora planteo, ¿Qué pasaría si Sophia empeora su estado actual, mejorando su óptimo global?. Algo que no ocurriría nunca con el pensamiento greedy, puede utilizarse a nuestro favor con un pensamiento cercano a la **Programación Dinámica**. Volviendo al ejemplo anterior, primer turno Sophia agarra la moneda de valor $[2]$, en vez de la mayor, "empeorando" su estado actual. Mateo, en su turno y de manera inocente, agarra la de mayor valor entre $[5, 1]$. En el turno de Sophia va a elegir entre $[100, 1]$. Eligiendo la $[100]$ y asegurándose la victoria + el mayor valor acumulado posible.

¿Cómo se puede aplicar esa técnica, para cualquier fila de monedas?

¿Existe un *patrón* que se cumpla siempre?

2.3. Consecuencias de la optimalidad

Si se logra pensar y crear una estrategia para que Sophia gane siempre que pueda y siempre obtenga el mayor valor acumulado posible, significa que:

- Las indicaciones propuestas fueron óptimas.
- Si pasamos las instrucciones a un algoritmo, ese mismo es óptimo.
- Si es posible demostrar un solo contraejemplo con respecto al mayor valor acumulado, significa que no era óptimo[1].
- Va a haber que demostrar matemáticamente que es óptimo y que no gana siempre.

Las pruebas a realizar de todo esto y mucho más se va a realizar en la lectura que sigue. Se recomienda leer detenidamente cada paso, y si no se entiende, releerlo. A eso sumarle el **pensamiento** tipo Programación Dinámica, hasta acá se dijo sutilmente como es éste. A continuación, se explica exáctamente eso: qué es un **algoritmo de Programación Dinámica**, qué técnica realiza, y más que nada cómo se puede implementar a este problema propuesto.

[1] La condición de que siempre obtenga el mayor valor acumulado, es obligatoria. No tiene por qué ganar siempre, ya hablaremos de esto en el futuro.

3. Teoría Programación Dinámica

"El pasado puede doler, pero tal como yo lo veo, puedes huir de él o aprender de él". - Rafiki (El Rey León, 1994).

Programación Dinámica es una técnica de programación (como los algoritmos Greedy). Ayuda a dejar de repetir cálculos aprovechando los ya hechos, usar resultados pasados para resolver el futuro. La solución por Programación Dinámica brinda resultados, estos resultados calculados los podemos guardar, lo que nos puede ayudar a optimizar el tiempo de ejecución. Esa técnica de guardado se llama *Memoization* o *Memorización*.

La Programación Dinámica explora implícitamente el espacio de posibles soluciones, lo cual siempre va a convenir ver el problema general en pequeños *subproblemas* del mismo. Nos basamos en la intuición que nos da la Memoization para reconocer los subproblemas y utilizarlos para construir la solución. Una vez que tenemos todas las soluciones memorizadas, el problema está resuelto. Nos ayuda evitar explorar un espacio *exponencial* de soluciones por fuerza bruta, y de esa manera podemos reducir la complejidad temporal del algoritmo a implementar.

Los *subproblemas* establecen una lógica inductiva:

- "Si yo puedo asumir que ya tengo calculados todas las soluciones anteriores (casos más pequeños), ¿Cómo puedo usarlos para construir la que necesito?"

Esto nos lleva a pensar en un cierto *patrón* que pueden generar los subproblemas y, en consecuencia, una *Ecuación de Recurrencia*[2] que cumpla con el armado de los subproblemas.

¿Qué pasaría si, buscando un valor anterior para encontrar la solución requerida, viene la persona más importante y que más sabe de este tema a decirte: "Esta es la solución que necesitas."? Confiás plenamente, pero ¿cómo utilizarías esta información a tu favor?

Ejemplo:

Secuencia de Fibonacci. Para un número n, no es necesario calcular todas las ramas de sus anteriores, porque se van a *repetir* resultados. Viene esta persona y te dice: "Para los 2 casos que quieras sumar, ya calculé los resultados y son estos". Entonces, simplemente hay que sumar esos dos *óptimos* que nos brindó, que sabemos que son correctos y no dudamos en ningún momento. ¿Cuál es la forma de los subproblemas? ¿Cuál es la ecuación de recurrencia?

$$\mathcal{OPT}(n) = \mathcal{OPT}(n-1) + \mathcal{OPT}(n-2)$$

Siendo $OPT(n)$ el resultado de fibonacci que queremos calcular.

Si $n = 5$, calcula $fibonacci(5) = fibonacci(4) + fibonacci(3)$

Entonces, si *iterativamente* tenemos precalculados, recordados, *memorizados* todos los resultados que queremos averiguar, los fibonacci anteriores a n, en una simple suma obtenemos el resultado requerido.

Para la resolución de los problemas utilizando la técnica de Programación Dinámica se prefiere mucho la forma iterativa (bottom-up), y no utilizar la forma recursiva (top-down). Similar a los algoritmos Greedy. Esto es debido a que:

[2] Una ecuación de recurrencia en el ámbito de la informática es una expresión matemática que define el tiempo de ejecución o el costo de un algoritmo en función de sí mismo para entradas más pequeñas.

1. Es mucho más fácil de entender qué esta pasando, cuándo y cómo.
2. Como consecuencia de lo anterior, se facilita el cálculo de la complejidad (temporal y espacial).
3. La resolución de problemas con esta técnica suelen ser más fáciles de resolver de forma inductiva que recursiva.
4. Nos estructura mucho más cómo pensar la solución[3].
5. Se pueden aplicar optimizaciones de memoria fácilmente.

¿Como se va a saber que puedo utilizar Programación Dinámica en un problema?

- Hay un número *polinomial* de subproblemas
- La solución al problema original puede ser construido a partir de soluciones a subproblemas
- Hay un orden natural de los subproblemas de menor a mayor. Los subproblemas "mayores" son resueltos mediante la composición de problemas "menores".

Otra utilidad que nos sirve usar la técnica de *Memoization*, comúnmente, permite pasar de un tiempo de ejecución exponencial a uno polinomial. Además, nos permite en ciertos ejercicios una optimización de memoria: en el anterior ejemplo de fibonacci, se puede utilizar un arreglo con todos los óptimos de 0 hasta n, para recordar las soluciones. Pero mejor aún, utilizar una optimización y guardar solamente 2 variables recordando las últimas dos soluciones. Eso nos va a brindar pasar de una complejidad espacial constante de $\mathcal{O}(n)$ a $\mathcal{O}(1)$.

Otra sección relacionada a la técnica de Programación Dinámica, es la *reconstrucción del problema*. Si yo quiero recordar y devolver todos los resultados del fibonacci en el orden en que los tuve que calcular, ahí sí se utiliza un arreglo con todos los óptimos precalculados y en una recorrida se pueden obtener qué resultados y *cuándo* los utilicé. La reconstrucción del problema es construir la solución de abajo hacia arriba.

3.1. Programación Dinámica aplicada al problema

En el contexto del juego de las monedas, el desafío de Sophia es planificar la mejor estrategia. El futuro es predecible, ya que sabemos que Mateo agarra siempre la moneda de mayor valor que se le cruce en frente. Entonces podemos predecir el comportamiento y las próximas filas de monedas en base a la moneda que agarremos en *cada turno*.

La decisión de Sophia en el turno actual no reduce el tamaño del problema en 1 moneda, sino en 2 monedas (cada ronda). Su propia elección sumada a la reacción avariciosa de Mateo.

El Pensamiento de Sophia:
En vez de elegir sin pensar la moneda de mayor valor (como haría un algoritmo Greedy), Sophia piensa mentalmente los dos universos posibles en su turno:

1. **¿Qué pasa si elijo la moneda de la izquierda?** Mateo, por instinto greedy, va a agarrar la mayor de las dos monedas que queden en los extremos. Sabiendo exactamente cuál va a elegir Mateo, Sophia se pregunta: *¿Cuántos puntos máximos me aseguro con las monedas que sobran en la mesa?*.

[3] Primero la ecuación de recurrencia, después la programación del problema (siendo este último paso, sencillo).

2. **¿Qué pasa si elijo la moneda de la derecha?** Mateo va a hacer exactamente lo mismo. Sophia vuelve a calcular: *En este otro caso, ¿cuántos puntos me aseguro para el futuro?*.

Sophia calcula el total de cada camino (la moneda que elige ahora + los puntos que ya sabe que va a ganar en el futuro) y ejecuta la opción que le devuelva el número más grande.

Ecuación de Recurrencia:
Para formalizar este pensamiento, definimos a $\mathcal{OPT}(\text{prim}, \text{ult})$ como el máximo valor acumulado que Sophia puede asegurarse en la fila de monedas actual.

Casos Base (y comúnmente, el final del juego):

- Si queda sólo una moneda: Sophia se la queda. $\mathcal{OPT}(\text{prim}, \text{prim}) = m[\text{prim}]$
- Si quedan dos monedas: Sophia elige la mayor. $\mathcal{OPT}(\text{prim}, \text{ult}) = \text{máx}(m[\text{prim}], m[\text{ult}])$

Caso General (Cuando quedan 3 o más monedas):
Lo mejor es simplemente el máximo entre esos dos futuros casos que calcula Sophia mentalmente:

$$\mathcal{OPT}(\text{prim}, \text{ult}) = \text{máx} \begin{cases} m[\text{prim}] + \text{Futuro si agarré la Primera} \\ m[\text{ult}] + \text{Futuro si agarré la Última} \end{cases}$$

Matemáticamente, ese "futuro" depende del Greedy aplicado de Mateo, quedando la ecuación completa así:

$$\mathcal{OPT}(\text{prim}, \text{ult}) = \text{máx} \begin{cases} m[\text{prim}] + \begin{cases} \mathcal{OPT}(\text{prim}+2, \text{ult}) & \text{si Mateo elige } m[\text{prim}+1] \\ \mathcal{OPT}(\text{prim}+1, \text{ult}-1) & \text{si Mateo elige } m[\text{ult}] \end{cases} \\ m[\text{ult}] + \begin{cases} \mathcal{OPT}(\text{prim}+1, \text{ult}-1) & \text{si Mateo elige } m[\text{prim}] \\ \mathcal{OPT}(\text{prim}, \text{ult}-2) & \text{si Mateo elige } m[\text{ult}-1] \end{cases} \end{cases}$$

Demostración de mayor valor acumulado posible:

Para demostrar que esta estrategia es la mejor que le podemos recomendar a Sophia y garantiza siempre el mayor valor acumulado posible para Sophia, nos basamos en el razonamiento inductivo.

Como vimos en los Casos Base, es un obvio que Sophia hace la decisión perfecta cuando quedan 1 o 2 monedas en la fila. El éxito de la estrategia de Programación Dinámica se basa en que, cuando Sophia se enfrenta a un problema de 3, 4 o n monedas, ella no improvisa. Al evaluar el "camino izquierdo" y el "camino derecho", la ecuación consulta los resultados de los subproblemas más chicos (el futuro). Ventaja superior comparado con el pensamiento Greedy de agarrar siempre la de mayor valor.

Como esos subproblemas del futuro fueron resueltos previamente construidos desde los casos base hacia arriba, iterativamente, Sophia está sumando su moneda actual con un futuro que es matemáticamente insuperable. Y como la ecuación evalúa el 100 % de las ramas posibles y siempre extrae la función máx() entre ellas, es lógicamente imposible que exista una combinación de elecciones en la fila que sume un valor mayor al que la estrategia da.

Se va a verificar, por demostración matemática y con ejemplos, en una sección a continuación.

3.2. Pensamiento y utilidades

El Pensamiento de Programación Dinámica es análisis. La mente comúnmente tiende a abrumarse con el futuro, esta mentalidad introduce: entender que los problemas gigantes no son bloques fijos, sino un árbol de subproblemas vinculados por el tiempo. Adoptar esta forma de razonar implica aprender a pensar de atrás hacia adelante. En lugar de pararse en el casillero inicial y avanzar a ciegas tanteando opciones a ver que onda, el "pensador dinámico" se para conceptualmente en la meta final que quiera y reconstruye el camino de forma inversa. Y lo mejor de todo, las personas hacen esto todo el tiempo en el día a día cuando planifican su vida, partiendo sus objetivos grandes en pequeñas acciones concretas hasta llegar a la primera decisión que tiene al alcance para hacer hoy. (Aristóteles hablando sobre la deliberación en Ética nicomáquea[4])

Esta forma de razonar retroespectiva tiene una base analítica muy fuerte: si el futuro está planificado de manera óptima, el presente se resuelve solo. Trasladado a las decisiones de la vida real, esto nos da una estructura clara para manejar la optimización del esfuerzo del día a día y la satisfacción aplazada a través de "ecuaciones de recurrencia personales". El que adopta este chip mental entiende que las decisiones actuales son variables que modifican el escenario del mañana. Por eso, se acepta de forma natural un costo, un esfuerzo extra o un rendimiento subóptimo en el plano inmediato (como aguantarse de salidas por estudiar o meterle horas a un proyecto) con la certeza matemática de que es el único camino válido para maximizar el resultado global a largo plazo. (Principio de Optimalidad de Bellman)

En un plano cognitivo, el enfoque PD introduce la *memoization* como una técnica estricta para guardar la experiencia personal. El pensamiento dinámico no deja el desperdicio del aprendizaje ni el desgaste de tropezar dos veces con la misma piedra. Si o si exige que cada crisis superada, cada error corregido y cada subproblema resuelto quede anotado en una "tabla mental" permanente. Al cruzarse con situaciones repetitivas en la rutina diaria, la mente no reinventa la lógica desde cero, sino que hace una consulta rápida a su memoria. Y así, la toma de decisiones cotidianas no permiten el azar y pasa a ser algo más razonable, donde lo vivido se transforma de forma directa en un mapa predictivo para resolver la incertidumbre.

[4]Expone en el Libro III de esta obra que la deliberación no empieza por el principio, sino por la meta. Analizamos los requisitos de fin a principio, descubriendo que "el último paso en la resolución es el primero en la ejecución".

4. Diseño del Algoritmo

¿Por qué Sophia piensa antes de actuar de manera avariciosa, y logra la mejor ganancia que pudo haber conseguido?

Si la estrategia hubiera omitido los posibles subproblemas, los posibles escenarios dependiendo de qué puede convenir elegir entre cada ronda, no se le está dando la manera de encontrar la mejor ganancia. Y más que justificada está la estrategia, pudiendo saber siempre en cada caso qué moneda Mateo va a elegir. No existe riesgo alguno con la estrategia a implmementar de Sophia.

Habiendo hablado sobre eso, hay que pasar esa estrategia, esa ecuación de recurrencia a la *lógica algorítmica*. ¿Cuál es el paso a paso en cualquier caso de monedas? ¿Cuál puede ser el peor caso posible? ¿Cómo interactúa el algoritmo con los datos del problema? ¿Cómo se pasan los datos del problema al idioma computacional?

4.1. Lógica

Programación Dinámica tiene la ventaja de que una vez pensada y hecha la *ecuación de recurrencia* se logra implementar el algoritmo de manera sencilla. Esto es debido a que se puede implementar la *memoization* con un arreglo (o matriz en este caso), y llenar esta matriz con iteraciones en las cuales adentro se implementa la lógica de la ecuación de recurrencia en cuestión.

Una fila de n monedas, se puede implementar con un arreglo de números enteros. Donde cada posición representa una moneda, con su valor asociado. El turno va a empezar siendo de Sophia, y es donde va a empezar la elección.

La lógica se gestiona utilizando una matriz `OPT` de longitud $n \times n$ para guardar el **óptimo** en cada caso. En primer instancia, se va a cubrir los casos bases (ya conocidos). Y después, se llena la matriz con cada caso, hasta la última moneda. En cada caso posible, siempre se elije la máxima ganancia para Sophia. Se empieza pensando en los subproblemas, en las posibles filas de `monedas` que pueden ir quedando en base a "qué pasaría si...". Empezando de menor cantidad de monedas, hasta llegar a la cantidad n de monedas y habiendo cubierto, teniendo la respuesta final.

También se muestra la **reconstrucción** del algoritmo, el por qué se responde de manera sencilla. Si le decimos a Sophia cuánto va a ser su ganancia máxima, la pregunta va a ser ¿Cómo logro esa ganancia? *¿Qué monedas se eligen, en cada caso?*

Con un manejo de índices, en base a la primer y última moneda, se les va a ir restando o sumando hasta que sean iguales (quede una sola moneda). Se resta 1 el índice `ultima` si se agarra esa moneda, y se suma 1 a `primera` si se agarra esa moneda en ese turno. Cada moneda que se va agarrando se guarda en un arreglo `CUALES`, que va a terminar teniendo longitud n (longitud de la fila de monedas).

Para saber qué moneda se elije (para el turno de Sophia, para Mateo no hace falta aclarar), se utiliza la misma ecuación de recurrencia pero *al revés*. ¿Qué hubiera pasado si en "este caso" hubiera hecho "esto"? La respuesta a esa pregunta, verificar el comportamiento que sucedió en esa ronda de elección, nos permite saber qué moneda se eligió.

4.2. Código

```
def juego(monedas [1 , 2 , 3 ,... , n ]):
    if len(monedas) == 0:
        Devuelvo (0, 0)

    n = len(monedas)

    OPT = [[0] * n for _ in range(n)]

    # casos base
    for i in range(n):
        OPT[i][i] = monedas[i] # posicion (i, i) a la moneda i
        if i < n -1:
            OPT[i][i+1] = max(monedas[i], monedas[i+1])

    # caso general
    for long in range(3, n+1):
        for i in range(n - long + 1):
            j = i + long -1
            # caso 1: S plantea elegir la primer moneda
            if monedas[i+1] > monedas[j]:
                # S agarra la moneda i. M agarra max. entre i+1 y j
                ganancia_izq = monedas[i] + OPT[i+2][j]
            else:
                ganancia_izq = monedas[i] + OPT[i+1][j-1]

            # caso 2: S plantea elegir la ultima moneda
            if monedas[i] > monedas[j-1]:
                # S agarra la moneda j. M agarra max. entre i y j-1
                ganancia_der = monedas[j] + OPT[i+1][j-1]
            else:
                ganancia_der = monedas[j] + OPT[i][j-2]

            OPT[i][j] = max(ganancia_izq, ganancia_der)

    guia = _reconstruccion(monedas, OPT)
    return partida(monedas, guia)



def reconstruccion(monedas [1 , 2 , 3 ,..., n], OPT [Matriz de optimalidad nxn]):
    CUALES = []
    primera = 0
    ultima = n -1

    turno_sophia = True

    while primera <= ultima:
        # mientras haya monedas para ver
        if turno_sophia:
            if primera == ultima:
                # una moneda, y turno S, indistinto cual agarrar
                CUALES.append(0)
                break

            # si S hubiera elegido la primera
```

```
56            if monedas[primera+1] > monedas[ultima]:
57                ganancia_izq = monedas[primera] + (OPT[primera+2][ultima] if (
    primera+2) <= j else 0) # siempre que se pueda elijo el optimo, sino 0
58            else:
59                ganancia_izq = monedas[primera] + (OPT[primera+1][ultima-1] if (
    primera+1) <= j-1 else 0)
60
61            #chequeo comportamiento gracias a la ecuacion de recurrencia al reves
62            if OPT[primera][ultima] == ganancia_izq:
63                CUALES.append(0)
64                primera += 1
65            else:
66                CUALES.append(1)
67                ultima -= 1
68        else:
69            # turno de M, piensa de manera Greedy, elije la mas grande disponible
    siempre que pueda.
70            if monedas[primera] > monedas[ultima]:
71                primera += 1
72            else:
73                ultima -= 1
74
75        # cambio de turno
76        turno_sophia = not turno_sophia
77
78    Devuelvo CUALES
```

5. Analizando el Algoritmo

Para analizar el algoritmo propuesto anteriormente, va a resultar útil plantear un ejemplo para analizar el comportamiento: Sea el arreglo de monedas $A = [96, 594, 437, 674, 950]$. Creamos la tabla o matriz para poder ir anotando los óptimos, ademas la llenamos de ceros.

indices	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0

- Una vez esta la matriz creada, vamos recorriendo cada moneda, la moneda 0 ira en la posición $(0, 0)$, la moneda 1 en la posición $(1, 1)$ y asi sucesivamente.
- Además, una vez tengamos la moneda i, en todos los pasos (excepto el final), vamos a comparar la moneda i con su siguiente, o sea $i + 1$, para asi ubicar la mayor de estas en la posición $(i + 1)$

indices	0	1	2	3	4
0	96	594	0	0	0
1	0	594	594	0	0
2	0	0	437	674	0
3	0	0	0	674	950
4	0	0	0	0	950

- En este caso podemos observar como para $i = 0$, en la posición $(0, 1)$ ubicamos al 594, ya que el mismo es mas grande que su anterior, el 96. Este mismo paso con todos las monedas (menos la ultima).
- Una vez tenemos la primer parte hecha, se prosigue a la segunda, donde hay que aplicar la ecuación de recurrencia. Al ser un algoritmo del tipo bottom-up (iterativamente), vamos a ir resolviendo el problema grande arrancando por los problemas mas chicos, esos empiezan utilizando unicamente tres monedas.
- Iniciamos con los subproblemas. Tenemos tres monedas y las mismas son $[96, 594, 437]$, ademas tenemos que $i = 0$ y $j = 2$. Primero evaluamos la ganancia que podríamos llegar a tener si elegimos la moneda de la izquierda, esta se compone de elegir el 96 y saber que Mateo elegirá la mas grande entre $[594, 437]$, por lo que nos quedaría como ganancia izquierda $= 96 + 437$, o lo que es lo mismo, $96 + OPT[0 + 2][2]$.
- Por otro lado, la ganancia derecha sera el resultado de tomar la ultima moneda, de valor 437, por lo que mateo se que elegirá la mayor entre $[594, 96]$ quedando nuevamente $437 + 96$, o $437 + OPT[0][2 - 2]$. Siendo que entonces el máximo aca es igual en ambos casos.

indices	0	1	2	3	4
0	96	594	533	0	0
1	0	594	594	0	0
2	0	0	437	674	0
3	0	0	0	674	950
4	0	0	0	0	950

- La distancia entre i y j sigue siendo de 3, pero ahora $i = 1$ y $j = 3$. Con esos datos analizamos la lista que contiene a $[594, 437, 674]$, inicialmente elegimos a 597, y Mateo va a elegir 674, por lo que hacemos $594 + OPT[1 + 1][3 - 1]$ (o sea 437), quedando una *ganancia izquierda* de 1031.
- Yendo hacia el final ahora, elegimos 674, Mateo elegirá 594 por ser mas grande que 437, entonces se vuelve a sumar $674 + OPT[1 + 1][2 - 1]$ (437), quedando una *ganancia derecha* de 1111 puntos. Siendo que entonces entre estos dos, Sophia va a elegir los 1111 puntos en la posición $OPT[1][3]$.

indices	0	1	2	3	4
0	96	594	533	0	0
1	0	594	594	1111	0
2	0	0	437	674	0
3	0	0	0	674	950
4	0	0	0	0	950

- Ahora $i = 2$ y $j = 4$, con la siguiente lista como material $[437, 674, 950]$. Tomando el elemento de la izquierda tenemos que, como Mateo tomara el 950 por sobre el 674, nos queda $437 + OPT[2 + 1][4 - 1]$ (674), siendo igual a 1111.
- Por el otro lado, si tomamos la ultima, o sea la moneda de valor 950, nos queda que, como Mateo tomara la de 674, sumamos $950 + OPT[2][4 - 2]$ (437) siendo igual a 1387. Tomando como máximo entre la ganancia por izquierda y por derecha, a la ganancia por derecha, guardando ese valor en $OPT[2][4]$.

indices	0	1	2	3	4
0	96	594	533	0	0
1	0	594	594	1111	0
2	0	0	437	674	1387
3	0	0	0	674	950
4	0	0	0	0	950

- Ahora si, el tamaño del subproblema crece, i pasa a estar a 3 números de j, por lo cual arrancamos con $i = 0$ y $j = 3$. Teniendo para trabajar la lista de $[96, 594, 437, 674]$. Si elegimos el 96, sabremos que Mateo va a elegir entre el 594 y el 674, manteniendo su decisión avariciosa de agarrar el 674, quedándonos $96 + OPT[0 + 1][3 - 1]$ (594) dando como *ganancia izquierda* 690 puntos.
- Del otro lado, tomando 674, Mateo va a elegir 437 por sobre 96, por lo que nos queda como *ganancia derecha* la cuenta de $674 + OPT[0][3 - 2]$ (594) siendo esto igual a 1268 puntos. Y como además es mayor al resultado anterior, nos quedaremos con esta opción y se guarda en $OPT[0][3]$.

indices	0	1	2	3	4
0	96	594	533	1268	0
1	0	594	594	1111	0
2	0	0	437	674	1387
3	0	0	0	674	950
4	0	0	0	0	950

- En este caso, $i = 1$ y $j = 4$, siendo la última iteración de este caso, donde ahora tenemos la lista $[594, 437, 674, 950]$.
- Arrancando con el lado izquierdo, elegimos el 594 y lo sumaremos al $OPT[1+1][4-1]$, ya que sabemos que Mateo se quedara con la última moneda. Esto nos da una *ganancia izquierda* de 1268.
- Para el otro caso, tomamos el 950 y nos fijamos que Mateo se quedará con el 674 por sobre el 594, entonces la estrategia queda como $950 + OPT[1][4-2]$ (594), quedando como *ganancia derecha* 1544, que al ser mayor a la ganancia izquierda, la guardaremos en $OPT[1][4]$.

indices	0	1	2	3	4
0	96	594	533	1268	0
1	0	594	594	1111	1544
2	0	0	437	674	1387
3	0	0	0	674	950
4	0	0	0	0	950

- Finalmente, en el ultimo paso, $i = 0$ y $j = 4$, acá tenemos toda la lista a disposición, por lo que comenzamos por calcular la ganancia izquierda, tomando al 96 y sumando $OPT[0+1][4-1]$ (1111), esto ya que Mateo con su pensamiento Greedy va a elegir 950 por sobre el 594, teniendo una *ganancia izquierda* de 1207 puntos.
- Elegimos el 950 y como el máximo entre 96 y 674 es 674, sumo $OPT[0][2]$ (533) y termina quedando como *ganancia derecha* 1483 puntos, siendo superior a la ganancia izquierda, por lo que para terminar de llenar la tabla, en $OPT[0][4]$ pondremos 1483.

indices	0	1	2	3	4
0	96	594	533	1268	1483
1	0	594	594	1111	1544
2	0	0	437	674	1387
3	0	0	0	674	950
4	0	0	0	0	950

Habiendo presentado este ejemplo, ahora se puede decir que se puede analizar el algoritmo implementado de varias formas, que vamos a estar explicando a continuación. Entre ellas, se va a hablar de la **complejidad temporal y espacial algorítmica** del algoritmo, y si es posible mejorarla.

También se habla de la optimalidad del algoritmo, anteriormente se hizo una breve presentación de ésta la cual ahora se va a explayar más y se va a demostrar matemáticamente por el método inductivo.

Por último se van a realizar experimentos sobre el mismo algoritmo, poniéndolo sobre arreglos de distintos tamaños para hacer y visualizar *mediciones* de tiempo distintos.

5.1. Complejidad

Cuando se habla de *complejidad algorítmica*, buscamos analizar el algoritmo de dos maneras: *espacial* y *temporal*. La complejidad espacial determina el espacio adicional ocupado en memoria por las estructuras de datos auxiliares, mientras que la complejidad temporal evalúa el tiempo de ejecución del algoritmo a medida que el tamaño de instancia (cantidad de monedas) crece. ¿Cómo se comporta el algoritmo? Para hacer un análisis justo, es clave separar el costo de rellenar la matriz de Programación Dinámica (`OPT`) por un lado, y el costo del algoritmo de reconstrucción por el otro. Se adopta el enfoque del *peor caso posible*[5], usándola notación Big-Oh ($\mathcal{O}$).

1. Complejidad de la Matriz (técnica Memoization):
El llenado de la matriz `OPT` depende de dos bucles `for` anidados. El bucle externo hace crecer la longitud de los subproblemas (desde 3 hasta n), y el interno recorre las posiciones de la fila de monedas. Si nos ponemos a contar, la cantidad total de celdas que calculamos es aproximadamente $\frac{n^2}{2}$ (mostrado en el ejemplo anterior). En cada una de esas celdas, el algoritmo hace operaciones matemáticas simples: accesos a posiciones de la matriz, sumas y sacar un máx(). Todo esto cuesta tiempo constante $\mathcal{O}(1)$.

Como calculamos $\frac{n^2}{2}$ celdas a un costo de $\mathcal{O}(1)$ cada una, la **complejidad temporal** de armar la matriz es $\mathcal{O}(n^2)$.

Por el lado de la **complejidad espacial**, como se usa la técnica de *Memoization* necesitamos guardar los resultados par ano recalcularlos. Como se crea la matriz de tamaño $n \times n$, lo que nos da un costo espacial de $\mathcal{O}(n^2)$.

2. Complejidad de la Reconstrucción:
La función `_reconstruccion` es más directa. Usa un solo `while` que va achicando la distancia entre los punteros `primera` y `ultima`. Adentro del bucle no se recalcula absolutamente nada, solo se consulta la matriz `OPT` (que cuesta $\mathcal{O}(1)$) para deducir qué camino eligió Sophia. Como en el peor de los casos da n pasos hasta que no quedan monedas, la **complejidad temporal** de reconstruir el camino es lineal, es decir $\mathcal{O}(n)$. El costo espacial acá es de $\mathcal{O}(n)$ para guardar la lista final con las decisiones hechas.

Complejidad final:

- **Complejidad temporal:** $\mathcal{O}(n^2) + \mathcal{O}(n) = \mathcal{O}(n^2)$
- **Complejidad espacial:** $\mathcal{O}(n) + \mathcal{O}(n) = \mathcal{O}(n)$

5.2. Demostración de la Optimalidad

Que el algoritmo planteado sea *óptimo* significa que la estrategia de Programación Dinámica le garantiza a Sophia obtener el máximo valor acumulado posible en cualquier escenario de monedas

[5] Es importante destacar que en este algoritmo de programación dinámica, la distribución o el orden de los valores de las monedas no altera el análisis (es la misma estrategia, misma ecuación de recurrencia). Por lo tanto, el peor caso, el mejor caso y el caso promedio son iguales en términos numéricos de operaciones ejecutadas, lo cual simplifica la validación experimental.

(y recordar que esto lleva a ganar siempre que pueda).

Para demostrar matemáticamente que esta ecuación de recurrencia es infalible, vamos a usar la **Demostración por Inducción**. Nos vamos a basar en la longitud del subarreglo de monedas que hay en la mesa, que se va a llamar L.

Hipótesis Inductiva $P(L)$: La función $\mathcal{OPT}(\text{prim}), \text{ult}$ devuelve el mayor valor acumulado posible para cualquier subarreglo de longitud L.

Casos Base ($L = 1$ y $L = 2$):
Como ya definimos en la ecuación de recurrencia (sección 3.1), si $L = 1$, Sophia se queda con la única moneda en la mesa (empieza ella), llegando al máximo obvio $\mathcal{OPT} = m[\text{prim}]$. Si $L = 2$, Sophia elige la mayor de las dos, asegurando el máximo $\mathcal{OPT} = \text{máx}(m[\text{prim}], m[\text{ult}])$. Ambos casos base son verdaderos.

Paso Inductivo:
Asumimos que la Hipótesis Inductiva $P(K)$ es verdadera para toda longitud de monedas $K < L$. El objetivo a lograr es demostrar que $P(L)$ también es verdadero para cualquier longitud de monedas $L \geq 3$.

Con una fila de longitud L, Sophia tiene que tomar una decisión en los extremos. Eso le abre exactamente dos caminos posibles:

1. **Camino Izquierdo (Elige la primera):** Mateo, avaricioso como siempre, va a agarrar la mayor de los extremos que queden en la mesa. Después de esto, la fila de monedas se achicó y ahora tiene una longitud exacta de $L - 2$. Por nuestra Hipótesis Inductiva, nosotros ya sabemos que el valor de $\mathcal{OPT}$ para este resto de longitud $L-2$ es el *máximo absoluto posible*. Entonces, la ganancia garantizada de este camino es la suma de $m[\text{prim}] + \mathcal{OPT}(\text{resto})$.

2. **Camino Derecho (Elige la última):** Mateo va a reaccionar igual, llevándose la mayor de los extremos disponibles. Y de vuelta, la fila queda de longitud $L - 2$. Por Hipótesis Inductiva, $\mathcal{OPT}(\text{resto})$ nos da el mejor puntaje para lo que sobra. La ganancia garantizada de este camino es $m[\text{ult}] + \mathcal{OPT}(\text{resto})$.

Como la **Programación Dinámica** se fija todo el espacio de soluciones posibles, la ecuación compara las dos ganancias (Izquierda y Derecha) y se queda si o si con la ganancia máxima (máx()). Como los subproblemas (el futuro) ya están garantizados como óptimos por la Hipótesis Inductiva, elegir la rama más pesada garantiza matemáticamente que la decisión para la longitud L también sea la óptima. Queda demostrado.

5.2.1. Impacto de la Variabilidad de los datos

¿Qué pasa si variamos la magnitud de los valores de las monedas (por ejemplo, mezclar monedas de valor 1 con otras de 1,000,000)? Vamos a analizar el impacto en dos partes clave sobre el problema: en los *tiempos del algoritmo* y en la *optimalidad*.

Impacto en los tiempos de ejecución
El algoritmo que llena la matriz de memoization es "ciego" a los valores de las monedas: sin importar qué tan grandes o raros sean los números, siempre va a calcular el 100 % de los subproblemas

válidos. Entonces, el tiempo de la matriz termina siendo $\Theta(n^2)$ [6]. Igualmente, la variabilidad sí podría tener un impacto (pequeño) en el algoritmo de *reconstrucción*. Dependiendo de si los valores de las monedas obligan a la CPU a tomar casi siempre la misma condición (por ejemplo, evaluar siempre como verdadero el ir por la izquierda), a nivel de hardware la computadora puede aprovechar la *branch prediction*[7], lo que acelera poco el tiempo real de ejecución de ese bloque. De todas formas sigue manteniéndose todo en $\mathcal{O}(n)$.

Impacto en la optimalidad (PD > Greedy):
Acá es donde la Programación Dinámica demuestra por qué es superior. Con una pura estrategia Greedy, la variabilidad (con una moneda grande después de la elección de Sophia) puede mostrar que el algoritmo de Sophia de ese estilo falla. Más que nada teniendo en cuenta que Mateo elige siempre la más grande, no hay que tender a la avaricia. Pero la Programación Dinámica es inmune a esto (explicado en un principio, utilizamos un ejemplo parecido). Un caso particular de variabilidad que lo muestra a la perfección: $\mathbf{A} = [\mathbf{2}, \mathbf{100}, \mathbf{4}, \mathbf{1}]$

- **Si Sophia juega Greedy:** Se fija en los extremos $[2, 1]$ y por ambición agarra el 2. Mateo evalúa $[100, 1]$ y se queda el 100. Sophia ve $[4, 1]$ y agarra el 4. Mateo se queda con el 1. *Resultado final:* Sophia suma 6, Mateo suma 11. **Sophia pierde.**
- **Si Sophia juega con Programación Dinámica:** Al pensar en todo el árbol de futuros posibles (subproblemas) con la ecuación de recurrencia, Sophia se da cuenta de que si agarra el 2, le va a dejar servido el 100 a Mateo. Entonces, sacrifica su óptimo local[8] y **elige el** 1. La fila queda $[2, 100, 4]$. Mateo (con su estrategia Greedy) agarra el 4. La fila se reduce a $[2, 100]$. Sophia, victoriosa, agarra el 100 y a Mateo le sobra el 2. *Resultado final:* Sophia suma 101, Mateo suma 6. **Sophia le gana a Mateo.**

Queda comprobado que la variabilidad de los datos arruina a los algoritmos Greedy, sin la visión del "Big Picture". La Programación Dinámica, al pre-calcular el costo de dejar monedas futuras evaluando todos los universos posibles en la mesa, **nunca empeora la optimalidad global** y asegura la victoria siempre que sea posible.

5.3. Experimentos

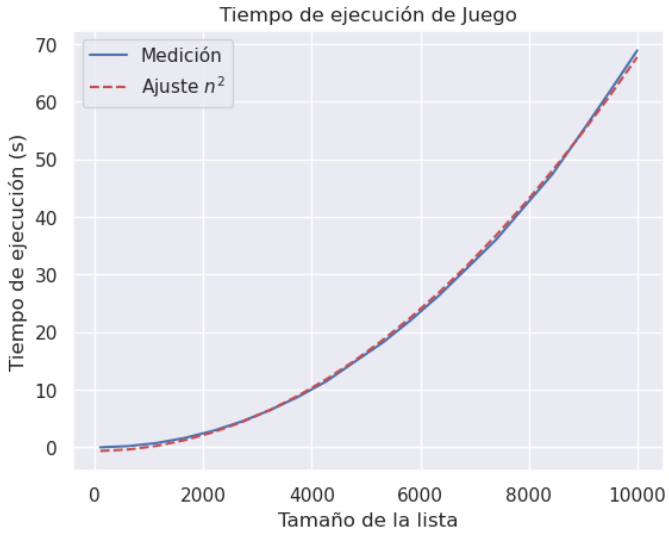
Se realizaron varios experimentos para demostrar el comportamiento del algoritmo planteado y su comparativa frente a otros tipos de algoritmos, para poder analizar su tiempo de ejecución en función de los arreglos o listas que reciba cada algoritmo, para ello contamos con dos sets de datos distintos, en uno se crearan veinte arreglos de tamaños que vayan desde cien elementos hasta diez mil.

En este primer set de datos podemos observar que en la Figura 1, las hipótesis eran ciertas respecto a pensar que el algoritmo planteado tendría una complejidad de $\mathcal{O}(n^2)$, ya que podemos observar una superposición prácticamente perfecta y exacta con la recta de ajuste por Cuadrados

[6] En la complejidad algorítmica, "Θ" representa la cota inferior y superior al mismo tiempo del algoritmo.
[7] Branch prediction o *predicción de saltos*, es un mecanismo de hardware donde el procesador "arriesga" y adelanta la ejecución del camino más probable en un condicional. Si el patrón de los datos es predecible, el procesador acierta siempre y el código corre más rápido. Si los datos (en este caso, la fila de monedas y su elección) son caóticos, los fallos de predicción ralentizan levemente la ejecución.
[8] Un algoritmo/estrategia/pensamiento Greedy *nunca* busca "empeorar" su óptimo local, para garantizar un mejor óptimo global.

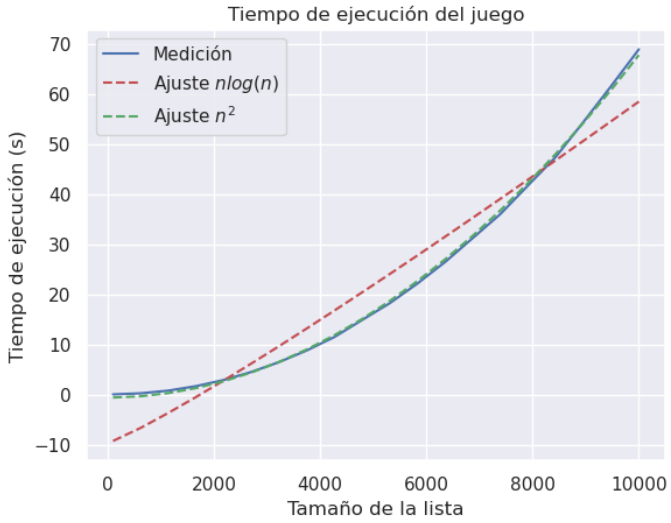
Mínimos[9], la cual sigue la trayectoria que debería tener un algoritmo cuadrático.

Figura 1: Tiempo de ejecución en función de la cantidad de elementos (n).



Aquí se sigue usando este mismo primer set de datos, pero en este experimento que se observa en la Figura 2 queríamos corroborar la diferencia de un algoritmo cuadrático, frente a uno que es lineal-logarítmico, donde se observa que si bien en una escasa parte del gráfico daría la sensación de que el algoritmo cuadrático es mejor frente a uno lineal-logarítmico, pero al instante se empieza a notar la pendiente tan pronunciada que tiene el algoritmo cuadrático, siendo que cuando se tenga un arreglo incluso mayor a diez mil elementos, la tendencia de la pendiente cuadrática va a ser cada vez mas grande respecto a la otra.

Figura 2: Comparación de un algoritmo cuadrático frente a uno $n.log(n)$.

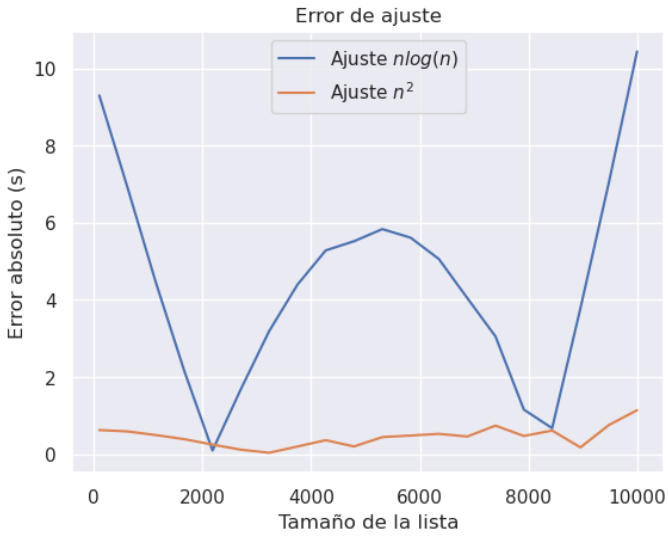


Además, si miramos con detenimiento ambas figuras de este mismo experimento (la Figura 2 y

[9]La técnica de Cuadrados Mínimos es un método de optimización matemática que, en este análisis, permite encontrar la función cuadrática que mejor se ajusta al conjunto de datos experimentales, *minimizando* la suma de los errores al cuadrado entre los *tiempos de ejecución* medidos y la curva $\mathcal{O}(n^2)$.

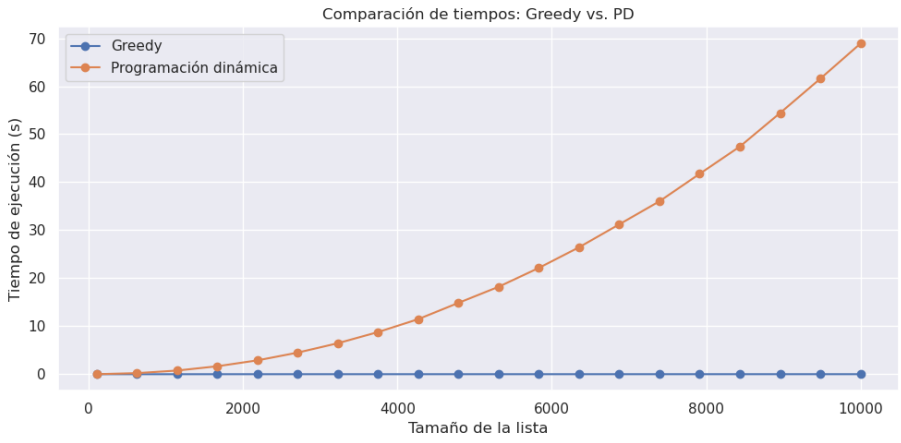
3), podemos notar que gracias al ajuste por Cuadrados Mínimos, las curvas prácticamente están siempre muy distantes entre si, a excepción de esos dos cruces entre las curvas de ambos algoritmos, eso explica el "arco" que se puede llegar a observar en la figura.

Figura 3: Error absoluto del ajuste por Cuadrados Mínimos.



Para la Figura 4, se sigue manteniendo el mismo set de antes, solo que ahora comparamos cuanto tiempo tarda el algoritmo Greedy que pudo haber utilizado Sophia, frente a lo que tarda el nuevo algoritmo con Programación Dinámica de Sophia, se puede ver cómo al principio van casi iguales, pero a penas empieza a crecer el tamaño de la lista se observa como la diferencia empieza a ser cada vez mayor, siendo que el algoritmo Greedy es una linea acostada ya que no muestra diferencia entre ejecutar una lista de pocos elementos frente a una de diez mil elementos, caso opuesto al algoritmo planteado para este problema, donde mientras el algoritmo greedy tarda un estimado de menos de un segundo para diez mil elementos, el algoritmo con programación dinámica tarda aproximadamente 70 segundos. Si dijéramos que el algoritmo greedy tardó medio segundo (por fijarlo en un valor), y el otro tardó setenta segundos, estaríamos hablando de que tarda 140 veces más que el algoritmo greedy.

Figura 4: Diferencia de tiempos entre greedy y programación dinámica.

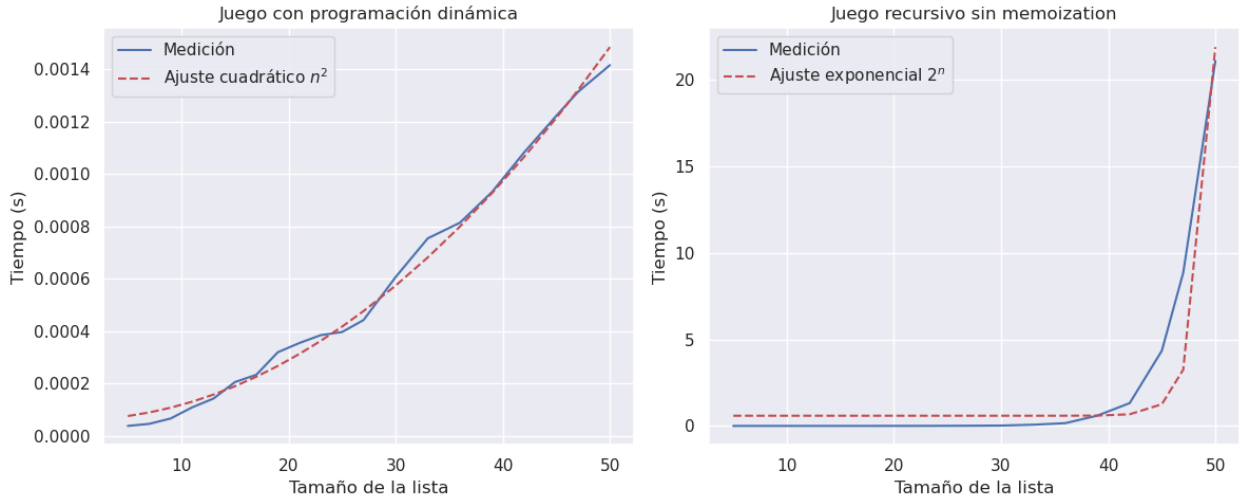


$$\frac{70 \text{ s}}{0{,}5 \text{ s}} = 140 \text{ veces más tiempo}$$

Finalmente, para este experimento (5) modificamos el set de datos, aquí no hay tamaños grandes ni mucho menos, debido a la complejidad del segundo algoritmo. En este caso se busca mostrar la diferencia entre dos algoritmos que esencialmente, hacen lo mismo, lo cual es hacer ganar a Sophia, incluso usan la misma ecuación de recurrencia, solo que uno lo hace de manera bottom-up con memoization (o sea que recuerda resultados previos), frente a otro que no guarda ningún tipo de resultado pre-calculado, por lo que recalcula las mismas cosas una y otra vez, y además es un algoritmo recursivo, con la técnica conocida como top-down. Justamente se busca mostrar como incluso algoritmos que dicen hacer lo mismo, el resultado es tan disparejo entre ambos, esto debido a que el algoritmo top-down tiene una complejidad de $\mathcal{O}(2^n)$, o sea, exponencial. Esto se traduce en que su tiempo de ejecución se dispara pasando cierto tamaño de listas, haciendo que mientras uno tome apenas unas pocas milésimas de segundo para un tamaño de 50 elementos, el otro puede llegar a tardar más de veinte segundos, llegando a ser catorce mil veces más lento en ese tamaño, luego empieza a ser cada vez más y más grande.

$$\frac{20 \text{ s}}{0{,}0014 \text{ s}} \approx 14{,}286 \text{ veces más tiempo}$$

Figura 5: Diferencia de tiempos entre algoritmo cuadrático y exponencial.



6. Conclusión

El desarrollo de este Trabajo Práctico nos permitió investigar y entender a fondo el poder de Programación Dinámica. Con el diseño de la ecuación de recurrencia y la matriz de memoization, se pudo resolver un escenario donde cada decisión presente desencadena un efecto dominó en el futuro, garantizando matemáticamente el máximo valor acumulado posible para Sophia en cada partida.

Al contrastar esta solución con el enfoque Greedy, mientras que solución avariciosa tenía un costo rapidísimo en $\mathcal{O}(n)$ usando memoria constante, falla al caer en ciertas "trampas" perdiendo la visión del problema completo frente a variabilidades grandes. En cambio, la Programación Dinámica exige pagar un precio alto: un sacrificio computacional cuadrático tanto en tiempo ($\mathcal{O}(n^2)$) como en espacio ($\mathcal{O}(n^2)$) para guardar la "foto o recuerdos" de la experiencia en una matriz. Pero nos da una garantía de optimalidad inquebrantable.

En conclusión, esta forma nos demuestra que la mejor solución global rara vez se encuentra eligiendo el camino más fácil y evidente en el presente. Resolver problemas complejos de verdad requieren la capacidad de sacrificar el beneficio inmediato, pensar en las ramificaciones del futuro, y tener la memoria suficiente para no recalcular los mismos errores del pasado.

Bibliografía

- Aristóteles (−330). *Ética nicomáquea (Libro III, 1112b)*. Atenas: Obra clásica. (330 a.C.)
- Bellman, Richard (1957). *Dynamic Programming*. Princeton, New Jersey: Princeton University Press.
- Buchwald, Martín y Genender Ezequiel (2026). *Teoría de Algoritmos (75.29) - Apuntes de cátedra y clases*. Facultad de Ingeniería, Universidad de Buenos Aires.
- Kleinberg, Jon y Éva Tardos (2005). *Algorithm Design*. Boston: Pearson Education.